

Design Note — HTB Cheatsheet Assistant (RAG)

1. Chunking Strategy

Failure of Naive Chunking on Penetration Testing Writeups

Fixed-size sliding windows (e.g., 512 tokens with overlap) fail catastrophically on offensive security walkthroughs:

1. **Command-Output Severing:** Slicing mid-sequence separates terminal commands from output banners, error strings, or extracted hashes, corrupting exploit context.
2. **Loss of Attack Lifecycle Phase:** Isolated commands (e.g., `secretsdump.py`) appear identical across enumeration, lateral movement, or domain compromise without structural heading context.
3. **Broken Code Blocks:** Splitting inside code fences produces orphaned triple backticks (`````), breaking Markdown parsers, embeddings, and LLM prompt templates.

The Solution: AST- & Heading-Aware Structural Chunking

Our chunker (`src/chunker.py`) segments along the offensive lifecycle (`## Phase $to$ ### Step $to$ #### Sub-step`) with deterministic guardrails:

- **Heading Normalization:** In 449 of 462 writeups, headings lacked preceding newlines (`Author ## Recon`). A regex pre-pass (`re.sub(r'([^\n#\\r])[ \t]*(#{2,4}[ \t]+[A-Za-z0-9])', r'\1\n\n2', raw)`) un-glues headings, recovering thousands of submerged steps.
- **Adaptive Segmentation:** Normal writeups split hierarchically at `###` (sub-splitting at `####` if `>2,400` chars / `approx 600` tokens). Monolithic writeups split recursively at paragraphs (`\n\n`) and lines (`\n`) under the strict 2,400-character ceiling.
- **Code Block Integrity:** When splitting oversized code blocks, the chunker cleanly terminates with `\n```` and reopens the next chunk with the identical language identifier (````bash\n`).
- **Context Injection (Semantic Breadcrumbs):** Every chunk is prepended with: `[Machine: <Name> | OS: <OS> | Phase: <Phase> | Path: <H2 > H3 > H4>]` This injects target identity and phase into dense and sparse representations, enabling accurate BM25 keyword matching and zero-overhead LLM citations.
- **Multimodal Screenshots:** Image markdown tags are stripped to remove URL noise; non-decorative exploit screenshots are captioned via Gemini Vision as text chunks (`chunk_type="image_desc"`).

2. Lexical vs. Embedding Retrieval & Hybrid RRF

Neither retrieval paradigm is sufficient in isolation for offensive security queries:

- **Lexical (BM25) Blindspot:** Fails on conceptual queries. "Windows privilege escalation cheatsheet" rarely matches writeups because practitioners document concrete actions (e.g., "abused `SelmpersonatePrivilege` via `PrintSpoofer`") rather than textbook category names.
- **Dense Embedding Blindspot:** Dense models (`all-MiniLM-L6-v2`) capture high-level semantics but struggle with exact, low-frequency technical tokens: CVE IDs (`CVE-2021-44228`), offensive tool names (`evil-winrm` , `certipy`), and CLI flags (`-just-dc` , `-k -no-pass`).

Multi-Stage Hybrid Pipeline

1. **Parallel Retrieval:** BM25 retrieves exact keywords, CVEs, and tool syntax; dense vector search captures conceptual and semantic intent.
2. **Reciprocal Rank Fusion (RRF, $k=60$):** Merges rank positions ($RRF(d) = \sum \frac{1}{60 + r_i(d)}$) rather than uncalibrated raw scores, preventing either retriever from dominating candidates.
3. **Cross-Encoder Neural Reranking (`ms-marco-MiniLM-L-6-v2`):** Jointly scores query-document token interactions across top 30 RRF candidates, filtering superficial lexical matches.
4. **Source Diversification:** Enforces per-machine chunk limits (1 for broad, 2 for specific queries), preventing single verbose writeups from flooding the context window.

3. The Motivation: Graph-Assisted Manifest Injection (Week 1 Analysis)

Our initial Week 1 empirical benchmark (`eval/results.md`) achieved **Precision: 0.67, Recall: 0.35**. While high-precision clusters excelled (Kerberoasting 1.00, WinRM 1.00, AD 0.96, ADCS 0.84), recall was severely constrained because broad queries have 60–200+ valid machines in the corpus, but LLM context windows restricted text chunk retrieval to $k=8$.

The Proposed Strategy

The repository contained a populated NetworkX knowledge graph (`graph/htb_graph.json`) mapping `Machine` `HAS_TECHNIQUE` `Technique` `BELONGS_TO` `Category` . The proposed architectural roadmap was:

1. **Intent-Based Graph Traversal:** In `src/retriever.py` , classify queries for broad cheatsheet intent and query the graph to extract all machines linked to the identified technique/category (e.g., all 22 ADCS machines or all 63 Active Directory machines).
2. **Prompt Manifest Injection:** Instead of retrieving 100 raw text chunks (causing context overflow), inject a compact **Machine Manifest Table** (Machine, OS, Difficulty, Technique) alongside the top 8 detailed text chunks.
3. **Dual-Layer Synthesis:** The LLM generates deep exploit steps from the 8 chunks, then cites comprehensive corpus coverage from the manifest:

"Demonstrated on Certified and Absolute; also featured across 20 other corpus machines including Authority, Escape, and Scepter."